Ex.No:1	EC2 CREATION
Date:	

AIM:

To create an EC2 instance in the AWS cloud, install Nginx to configure it as a web server, and modify the default web page to display "Hello World."

PROCEDURE:

1. Understanding EC2 and AWS Global Infrastructure

AWS provides Elastic Compute Cloud (EC2), a service for launching virtual machines.

- AWS Global Infrastructure includes Regions and Availability Zones (AZs):
  - **Region:** A geographical area (e.g., US-East-1 in Virginia).
  - **Availability Zones (AZs):** Multiple isolated data centers within a region.
- **EC2 Components:**
  - **Amazon Machine Image (AMI):** Pre-configured OS images for instances.
  - **Instance Type:** Defines CPU, memory, and storage.
  - **SSH & SSH Keys:** Secure remote access to the instance.
  - **Linux Commands:** Used to configure the server.
  - **Web Server (Nginx):** Serves web pages on the internet.

2. Creating an EC2 Instance

Step 1: Login to AWS and Navigate to EC2

a. Go to AWS Management Console: AWS Console
b. Search for EC2 in the AWS Services and open it.
c. Click on **Launch Instance**.

Step 2: Configure Instance Details

i. **Name:** Enter a name for your instance (e.g., "MyWebServer").
ii. **AMI (Amazon Machine Image):** Select **Ubuntu 22.04 LTS**.
iii. **Instance Type:**
    1. Choose **t2.micro** (eligible for free tier).

iv. **Key Pair:**
    1. Click **Create new key pair** (if not available).
    2. Choose **RSA**, download the private key (**.pem** file), and save it securely.

v. **Security Group Configuration:**
    1. Allow **SSH (port 22)** for remote access.
    2. Allow **HTTP (port 80)** to serve web content.

vi. Click **Launch the Instance**.

3. Connecting to EC2 via SSH

a. Open a terminal (**Linux/macOS**) or use **PuTTY** (**Windows**).
b. Change the permissions of the key:

```
chmod 400 your-key.pem
```

c. Connect to the EC2 instance:

```
ssh -i your-key.pem ubuntu@<EC2-Public-IP>
```

4. Installing and Configuring Nginx

a. Update package lists:

```
sudo apt update
```

b. Install Nginx:

```
sudo apt install nginx -y
```

c. Start the Nginx service:

```
sudo systemctl start nginx
```

d. Enable Nginx to start on boot:

```
sudo systemctl enable nginx
```

5. Changing the Default Web Page

1. Navigate to the web server's root directory:

```
cd /var/www/html
```

2. Modify the default **index.html** file:

```
sudo nano index.html
```

3. Replace the content with:

```
<html>
    <head><title>My Web Server</title></head>
    <body><h1>Hello World</h1></body>
</html>
```

4. Save and exit (**CTRL+X, then Y, then ENTER**).

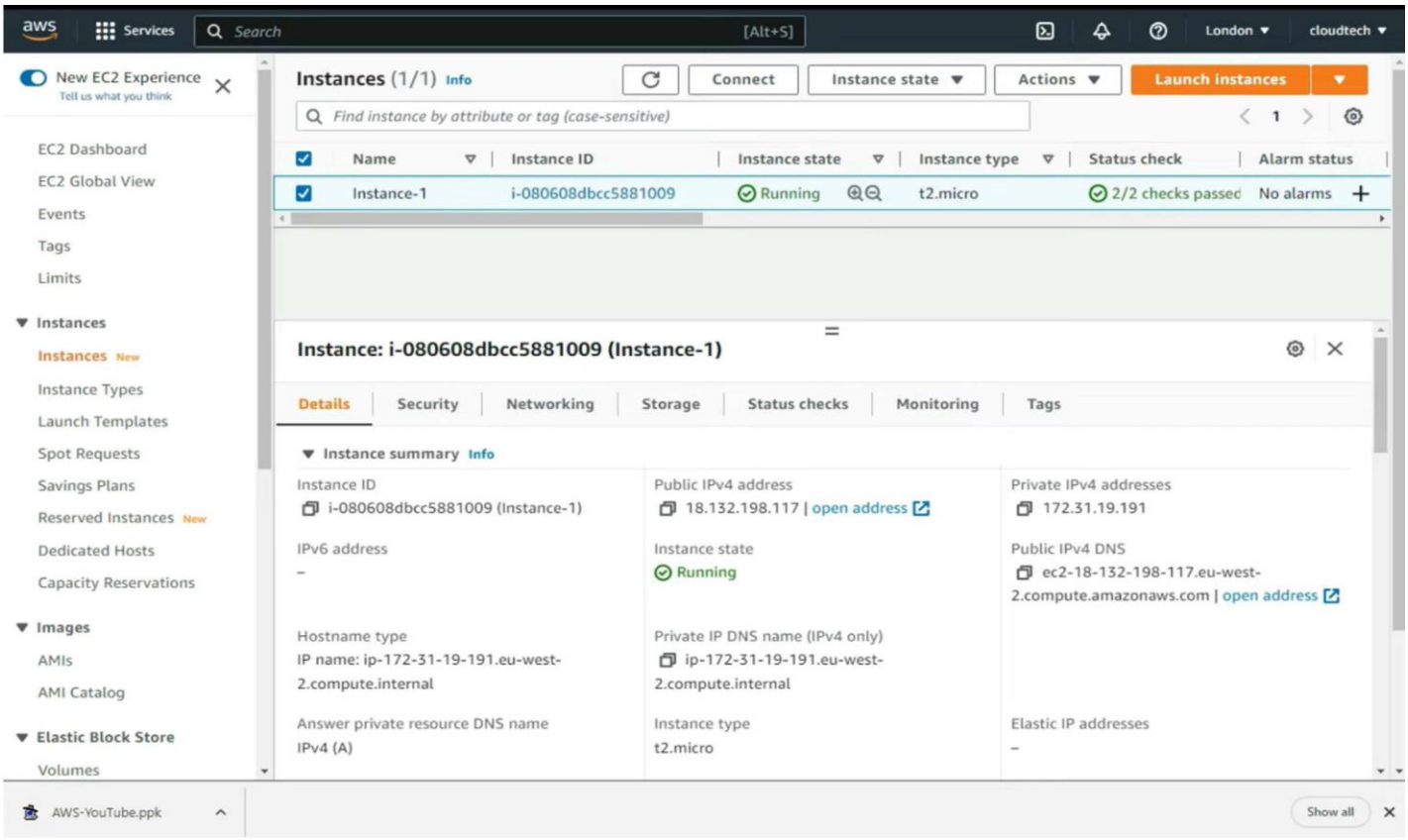
6. Verifying the Web Server

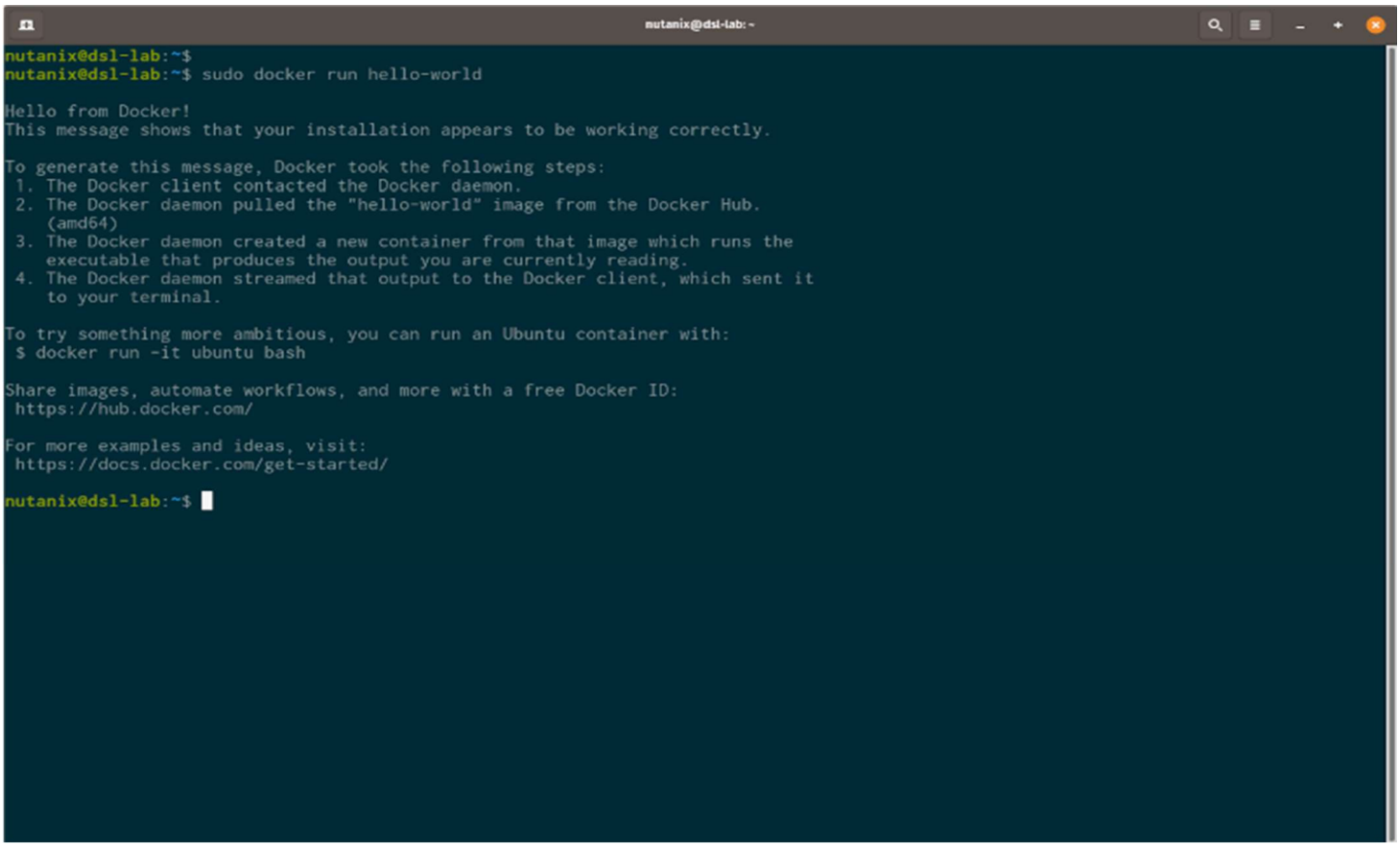
1. Get the **public IP** of your instance from the EC2 dashboard.
2. Open a browser and enter:

   http://<EC2-Public-IP>

3. The page should display **"Hello World"**.

OUTPUT:


aws
Services
Search
[Alt+S]
London
cloudtech
New EC2 Experience
Tell us what you think
EC2 Dashboard
EC2 Global View
Events
Tags
Limits
Instances
Instances New
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances New
Dedicated Hosts
Capacity Reservations
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Instances (1/1) Info
Connect
Instance state
Actions
Launch instances
Find instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Alarm status
Instance-1
i-080608dbcc5881009
Running
t2.micro
2/2 checks passed
No alarms
Instance: i-080608dbcc5881009 (Instance-1)
Details
Security
Networking
Storage
Status checks
Monitoring
Tags
Instance summary Info
Instance ID
i-080608dbcc5881009 (Instance-1)
Public IPv4 address
18.132.198.117 | open address
Private IPv4 addresses
172.31.19.191
IPv6 address
–
Instance state
Running
Public IPv4 DNS
ec2-18-132-198-117.eu-west-2.compute.amazonaws.com | open address
Hostname type
IP name: ip-172-31-19-191.eu-west-2.compute.internal
Private IP DNS name (IPv4 only)
ip-172-31-19-191.eu-west-2.compute.internal
Answer private resource DNS name
IPv4 (A)
Instance type
t2.micro
Elastic IP addresses
–
AWS-YouTube.ppk
Show all


nutanix@dsl-lab: ~
nutanix@dsl-lab:~$
nutanix@dsl-lab:~$ sudo docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the
executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
to your terminal.
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
For more examples and ideas, visit:
https://docs.docker.com/get-started/
nutanix@dsl-lab:~$

Ex.No:2	EBS CREATION
Date:	

AIM:

To create an Elastic Block Store (EBS) volume in AWS, attach it to an existing EC2 instance, and resize the volume to ensure it reflects in the connected instance.

PROCEDURE:

1. Understanding EBS

Elastic Block Storage (EBS)

- A persistent storage service that provides block-level storage for EC2 instances.
- Data remains even after the instance is stopped or terminated (unless deleted manually).

Volume Types

- General Purpose SSD (gp3, gp2): Balanced cost and performance.
- Provisioned IOPS SSD (io1, io2): High-performance applications.
- Magnetic (st1, sc1): Cost-effective for archival data.

Key Features

- Volume Creation: Independent of EC2 instances.
- Volume Snapshot: Backup mechanism for EBS volumes.
- Volume Attachment: EBS volumes can be attached to any running EC2 instance.

2. Creating and Attaching EBS Volume

Step 1: Launch a Linux EC2 Instance

1. Go to AWS Console → EC2 Dashboard.
2. Click Launch Instance and configure:
   - Name: "MyEBSInstance"
   - AMI: Ubuntu 22.04 LTS
   - Instance Type: t2.micro (Free Tier)
   - Key Pair: Create or use an existing one.
   - Security Group: Allow SSH (Port 22)
3. Click Launch.
4. Note down the Instance ID and Public IP Address.

Step 2: Create an EBS Volume (20GB)

1. Go to AWS Console → EC2 Dashboard → Elastic Block Store → Volumes.
2. Click Create Volume and set:
   - Size: 20 GB
   - Volume Type: gp3 (General Purpose SSD)
   - Availability Zone: Must match the EC2 instance's AZ (e.g., us-east-1a).
3. Click Create Volume.

Step 3: Attach the EBS Volume to EC2

1. Go to EC2 → Elastic Block Store → Volumes, select the created volume.
2. Click Actions → Attach Volume.
3. Select the EC2 instance from the list.
4. Note down the device name (e.g., /dev/xvdf).
5. Click Attach Volume.

Step 4: Verify the Volume on the EC2 Instance

1. Connect to EC2 via SSH:

   sh
   Copy code
   ssh -i your-key.pem ubuntu@<EC2-Public-IP>

2. Check attached block devices:

   sh
   Copy code
   lsblk

- Output should list the EBS volume (/dev/xvdf) as an unformatted disk.

Step 5: Format and Mount the Volume

1. Create a file system:

   sh
   Copy code
   sudo mkfs -t ext4 /dev/xvdf

2. Create a mount directory:

   sh
   Copy code
   sudo mkdir /mnt/ebs-volume

3. Mount the volume:

   sh
   Copy code
   sudo mount /dev/xvdf /mnt/ebs-volume

4. Verify by checking disk space:

   sh
   Copy code
   df -h

- The 20GB volume should be listed under /mnt/ebs-volume.

5. Make the mount persistent (Optional):

   - Edit the fstab file:
     sh
     Copy code
     sudo nano /etc/fstab

   - Add the following line:
     bash
     Copy code
     /dev/xvdf /mnt/ebs-volume ext4 defaults,nofail 0 2

   - **Save and exit (CTRL+X, then Y, then ENTER).**

3. Resizing the Attached EBS Volume

Step 1: Modify the Volume Size in AWS Console

1. Go to EC2 → Elastic Block Store → Volumes.
2. Select the attached volume.
3. Click Modify Volume.
4. Change the size from 20GB to 30GB.
5. Click Modify → Confirm the action.

Step 2: Reflect Changes in the EC2 Instance

1. Check the new volume size:
   sh
   Copy code
   lsblk

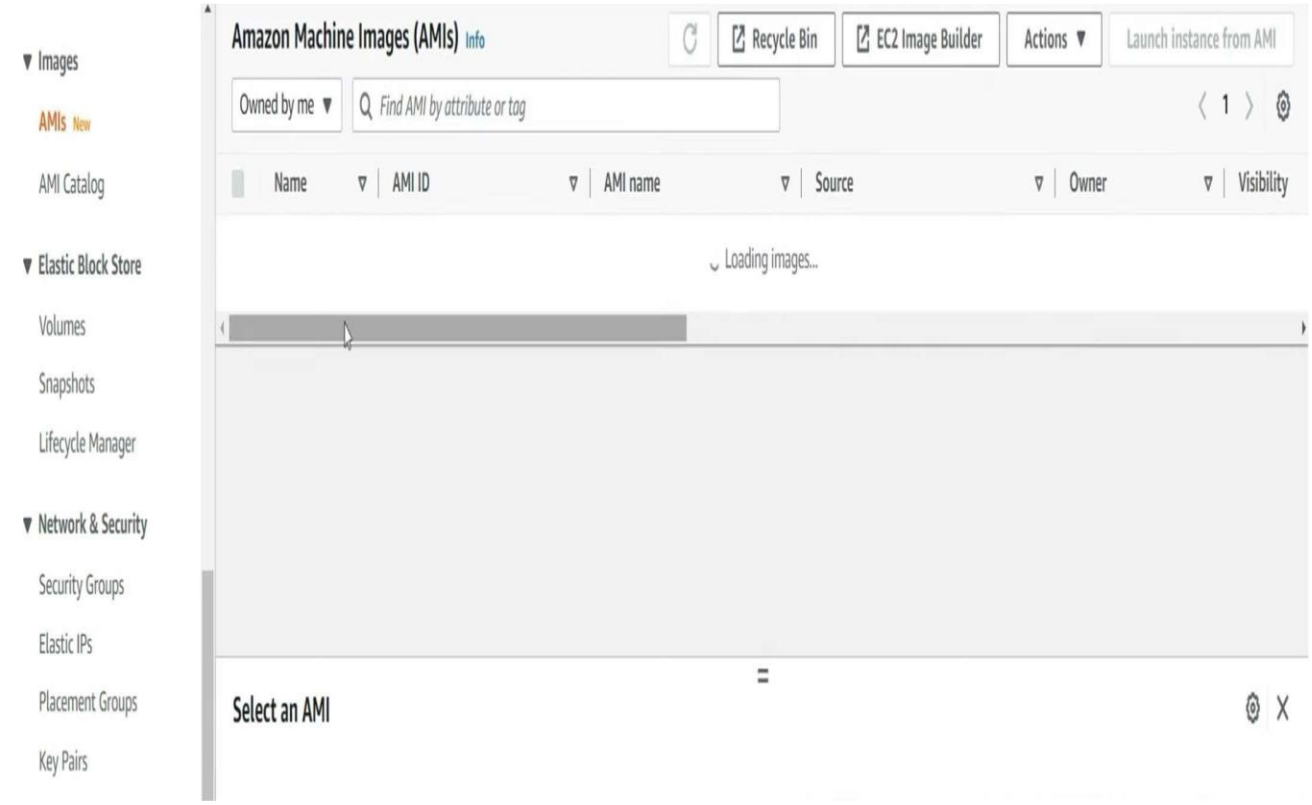
- The volume will show 30GB, but the file system is still 20GB.

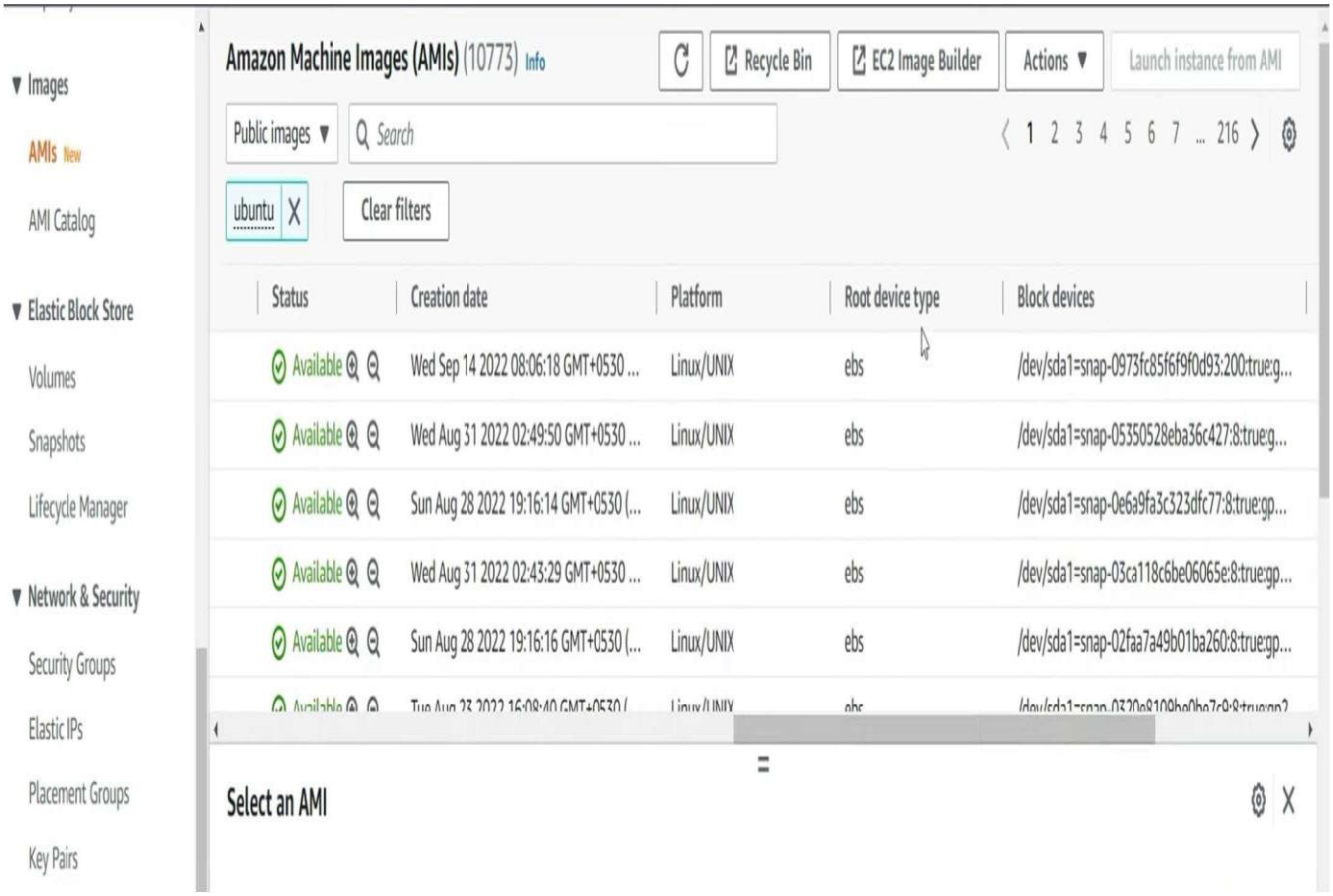
2. Resize the file system:
   sh
   Copy code
   sudo resize2fs /dev/xvdf

3. Verify the new size:
   sh
   Copy code
   df -h

- The mount should now reflect 30GB.

OUTPUT:


Images
AMIs New
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Amazon Machine Images (AMIs) Info
Recycle Bin
EC2 Image Builder
Actions
Launch instance from AMI
Owned by me
Find AMI by attribute or tag
Name
AMI ID
AMI name
Source
Owner
Visibility
Loading images...
Select an AMI


Images
AMIs New
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Amazon Machine Images (AMIs) (10773) Info
Recycle Bin
EC2 Image Builder
Actions
Launch instance from AMI
Public images
Search
1 2 3 4 5 6 7 ... 216
ubuntu
Clear filters
Status
Creation date
Platform
Root device type
Block devices
Available
Wed Sep 14 2022 08:06:18 GMT+0530 ...
Linux/UNIX
ebs
/dev/sda1=snap-0973fc85f6f9f0d93:200:true:g...
Available
Wed Aug 31 2022 02:49:50 GMT+0530 ...
Linux/UNIX
ebs
/dev/sda1=snap-05350528eba36c427:8:true:g...
Available
Sun Aug 28 2022 19:16:14 GMT+0530 (...
Linux/UNIX
ebs
/dev/sda1=snap-0e6a9fa3c323dfc77:8:true:gp...
Available
Wed Aug 31 2022 02:43:29 GMT+0530 ...
Linux/UNIX
ebs
/dev/sda1=snap-03ca118c6be06065e:8:true:gp...
Available
Sun Aug 28 2022 19:16:16 GMT+0530 (...
Linux/UNIX
ebs
/dev/sda1=snap-02faa7a49b01ba260:8:true:gp...
Select an AMI

Ex.No:3	EFS CREATION
Date:	

AIM:

To create an Elastic File System (EFS) in AWS and connect it to three different EC2 instances running different operating systems: **Ubuntu, Red Hat Linux, and Amazon Linux**.

PROCEDURE:

1. Understanding EFS

Amazon Elastic File System (EFS)

- A fully managed shared file system that can be accessed by multiple EC2 instances.
- Automatically scales storage capacity as needed.
- Provides high availability across multiple Availability Zones.

Difference Between EBS and EFS

Feature	EBS (Elastic Block Store)	EFS (Elastic File System)
Storage Type	Block Storage	Shared File Storage
Access	Single EC2 instance (unless attached as multi-attach)	Multiple EC2 instances
Scaling	Manual resizing required	Automatically scalable
Use Case	Databases, application storage	Shared file system, web servers

2. Creating and Mounting EFS

Step 1: Create an EFS File System

1. Navigate to **AWS Console → EC2 Dashboard → Elastic File System (EFS)**.
2. Click **Create File System**.
3. Configure the following:
    - **Name**: MyEFS
    - **Virtual Private Cloud (VPC)**: Select the same VPC as your EC2 instances.
    - **Storage Class**: Standard

- **Performance Mode**: General Purpose
- **Throughput Mode**: Bursting

4. Click **Next**, review settings, and click **Create File System**.
5. After creation, note the **File System ID** (e.g., fs-xxxxxxxx).

Step 2: Configure EFS Security and Mount Target

1. In the **EFS Console → File System → MyEFS**, click **Network Access**.
2. Click **Manage Mount Targets** and ensure EFS is available in all required Availability Zones.
3. Update the **Security Group** to allow **NFS traffic (port 2049)** from EC2 instances.

Step 3: Launch Three EC2 Instances

1. Navigate to **AWS Console → EC2 → Launch Instance**.
2. Create three instances:
   - **Instance 1**: Ubuntu
   - **Instance 2**: Red Hat Linux
   - **Instance 3**: Amazon Linux
3. Ensure all instances are in the same **VPC and subnet** as the EFS mount targets.
4. Allow **SSH (port 22)** and **NFS (port 2049)** in the **Security Group**.
5. Click **Launch**.

Step 4: Mount EFS on Each EC2 Instance

1. Connect to each instance via SSH:

bash
Copy code
```
ssh -i your-key.pem ubuntu@<EC2-Public-IP>
```

3. Install the NFS client (required for mounting EFS):

- **Ubuntu:**
  bash
  Copy code
  ```
  sudo apt update
  sudo apt install nfs-common -y
  ```
- **Red Hat Linux:**
  bash
  Copy code
  ```
  sudo yum install nfs-utils -y
  ```
- **Amazon Linux:**
  bash
  Copy code
  ```
  sudo yum install nfs-utils -y
  ```

4. **Create a mount directory:**

   bash
   Copy code
   sudo mkdir -p /mnt/efs

5. **Mount the EFS volume:**

   bash
   Copy code
   sudo mount -t nfs4 <EFS-DNS-Name>:/ /mnt/efs

- Replace <EFS-DNS-Name> with the **EFS mount target** (found in AWS EFS console, e.g., fs-xxxxxxxx.efs.us-east-1.amazonaws.com).

6. **Verify the mount:**

   bash
   Copy code
   df -h

- The **EFS file system** should appear in the output.

Step 5: Test File Sharing Across Instances

1. Create a test file on one instance:

bash
Copy code
echo "Hello from EFS!" | sudo tee /mnt/efs/testfile.txt

2. Verify file availability on other instances:

bash
Copy code
cat /mnt/efs/testfile.txt

- **Expected Output:**

  bash
  Copy code
  Hello from EFS!

- This confirms that all instances are **sharing the same storage**.

OUTPUT:

```
root@ip-172-31-35-13:/efs
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File1
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File2
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File3
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File5
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File6
drwxr-xr-x  2 root root      6144 Mar 10 21:39 Labs
[root@ip-172-31-35-13 efs]# nano Test1
[root@ip-172-31-35-13 efs]# ls -al
total 2934032
drwxr-xr-x  4 root root      6144 Mar 10 21:44 .
dr-xr-xr-x 19 root root       268 Mar 10 18:53 ..
-rw-r--r--  1 root root   1265445 Mar  9 19:32 aws-cli.pdf
drwxr-xr-x  2 root root      6144 Mar 10 21:39 Documentation
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File1
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File2
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File3
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File5
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File6
drwxr-xr-x  2 root root      6144 Mar 10 21:39 Labs
-rw-r--r--  1 root root        20 Mar 10 21:44 Test1
[root@ip-172-31-35-13 efs]# chmod 777 Test1
[root@ip-172-31-35-13 efs]# ls -al
total 2934032
drwxr-xr-x  4 root root      6144 Mar 10 21:44 .
dr-xr-xr-x 19 root root       268 Mar 10 18:53 ..
-rw-r--r--  1 root root   1265445 Mar  9 19:32 aws-cli.pdf
drwxr-xr-x  2 root root      6144 Mar 10 21:39 Documentation
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File1
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File2
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File3
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File5
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File6
drwxr-xr-x  2 root root      6144 Mar 10 21:39 Labs
-rwxrwxrwx  1 root root        20 Mar 10 21:44 Test1
[root@ip-172-31-35-13 efs]#
```

```
ec2-user@ip-172-31-47-208:/efs
login as: ec2-user
Authenticating with public key "imported-openssh-key"
Last login: Wed Mar 10 21:41:17 2021 from 52.95.4.11

       __|  __|_  )
       _|  (     /   Amazon Linux 2 AMI
      ___|\___|___|

https://aws.amazon.com/amazon-linux-2/
No packages needed for security; 2 packages available
Run "sudo yum update" to apply all updates.
[ec2-user@ip-172-31-47-208 ~]$ cd /
[ec2-user@ip-172-31-47-208 /]$ cd efs
[ec2-user@ip-172-31-47-208 efs]$ ls -al
total 2934028
drwxr-xr-x  4 root root      6144 Mar 10 21:39 .
dr-xr-xr-x 19 root root       268 Mar 10 21:31 ..
-rw-r--r--  1 root root   1265445 Mar  9 19:32 aws-cli.pdf
drwxr-xr-x  2 root root      6144 Mar 10 21:39 Documentation
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File1
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File2
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File3
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File5
-rw-r--r--  1 root root 600630678 Mar  9 19:32 File6
drwxr-xr-x  2 root root      6144 Mar 10 21:39 Labs
[ec2-user@ip-172-31-47-208 efs]$
```

```
root@ip-172-31-35-13:/efs
  GNU nano 2.9.8                    Test1                        Modified
This is a test file

Save modified buffer?  (Answering "No" will DISCARD changes.)
 Y Yes
 N No           ^C Cancel
```

Ex.No:4	S3 BUCKET CREATION
Date:	

AIM:

To create an Amazon S3 bucket for file storage, upload five different objects with various file extensions, and understand bucket properties and data transfer in AWS.

PROCEDURE:

1. Understanding S3 Concepts

Amazon S3 (Simple Storage Service):

- Object storage that stores data as objects within buckets.
- Highly scalable, secure, and durable cloud storage service.
- Each object consists of data, metadata, and a unique key (filename).

S3 Bucket Properties:

- **Public or Private Access** – Controls who can read/write objects.
- **Versioning** – Keeps multiple versions of an object.
- **Lifecycle Policies** – Automatically transitions objects between storage classes.
- **Storage Classes:**
  - **S3 Standard** (default)
  - **S3 Infrequent Access (IA)**
  - **S3 Glacier** (for archival storage)

Data Transfer in S3:

- Upload via AWS Console, AWS CLI, or SDKs.
- **Multipart Upload** – Optimized for large files.

2.Creating and Uploading Files to an S3 Bucket

Step 1: Create an S3 Bucket

1. Go to **AWS Console → S3 Dashboard**.
2. Click **Create Bucket**.
3. Configure the bucket with the following settings:
   - **Bucket Name**: my-s3-bucket-dhanush *(must be globally unique)*
   - **Region**: Choose your preferred AWS region.
   - **Block Public Access**: Enabled (for security).
   - **Versioning**: Enabled (optional).
   - **Encryption**: Enable **SSE-S3** (for security).
4. Click **Create Bucket**.

Step 2: Upload Files to S3

1. Open your created **S3 bucket**.
2. Click **Upload → Add files**.
3. Select five different files from your computer with different extensions, for example:
   - **document.pdf** *(PDF file)*
   - **image.png** *(Image file)*
   - **video.mp4** *(Video file)*
   - **data.csv** *(CSV file)*
   - **script.py** *(Python script)*
4. Click **Upload**.

Step 3: Verify and Manage Uploaded Files

1. Check if all five files are listed in the bucket.
2. Click on a file to view details like:
   - **Object URL**
   - **Metadata**
   - **Permissions**
3. Try downloading a file to ensure it is stored correctly.

Step 4: Access Files via AWS CLI (Optional)

1. Install and Configure AWS CLI

Run the following command to configure AWS CLI:

sh
Copy code
aws configure

Provide the following details:

- AWS **Access Key**
- AWS **Secret Key**
- AWS **Region**
- **Output format**

2. Upload a File via AWS CLI

Use the following command to upload a file:

sh
Copy code
aws s3 cp /path/to/document.pdf s3://my-s3-bucket-dhanush/

3. List Objects in the Bucket

To view uploaded files, use:

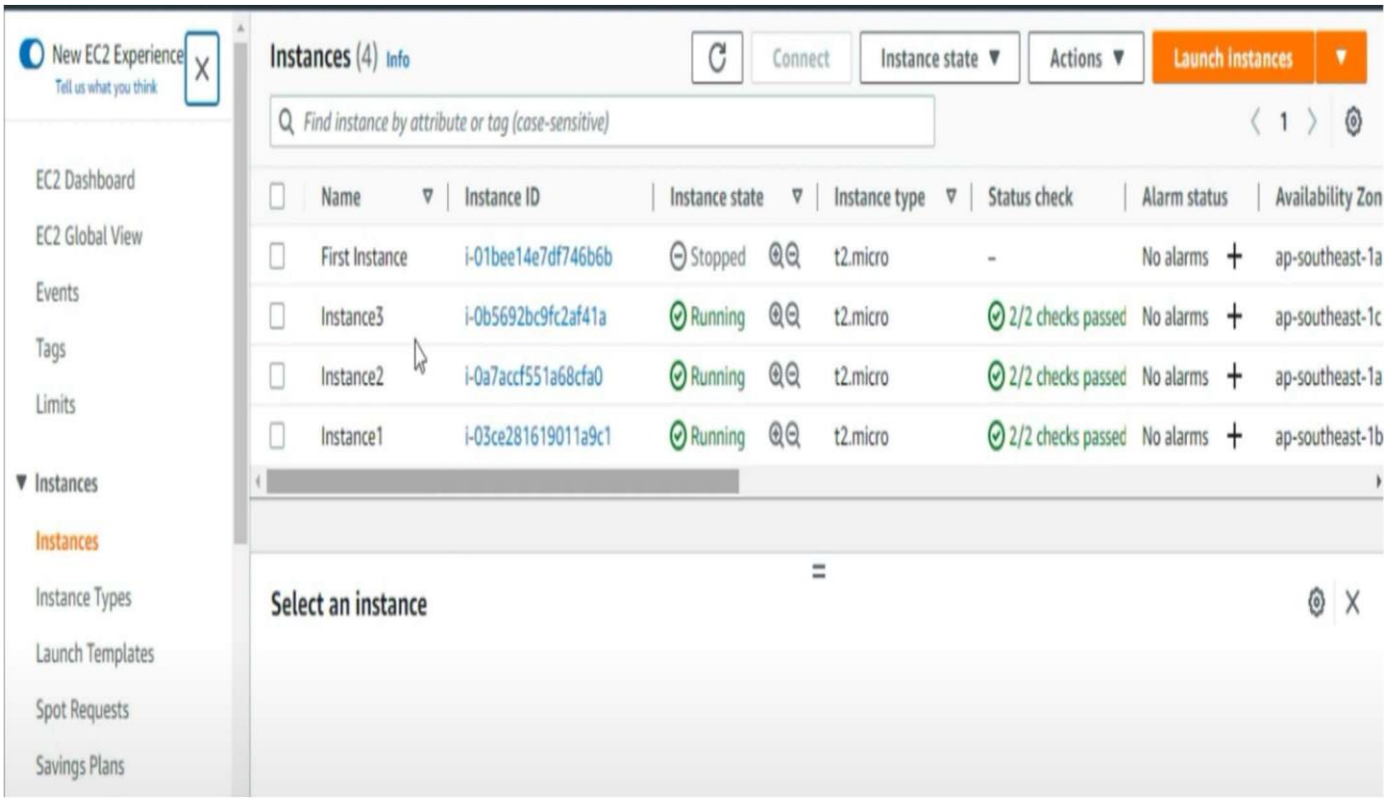
sh
Copy code
aws s3 ls s3://my-s3-bucket-dhanush/

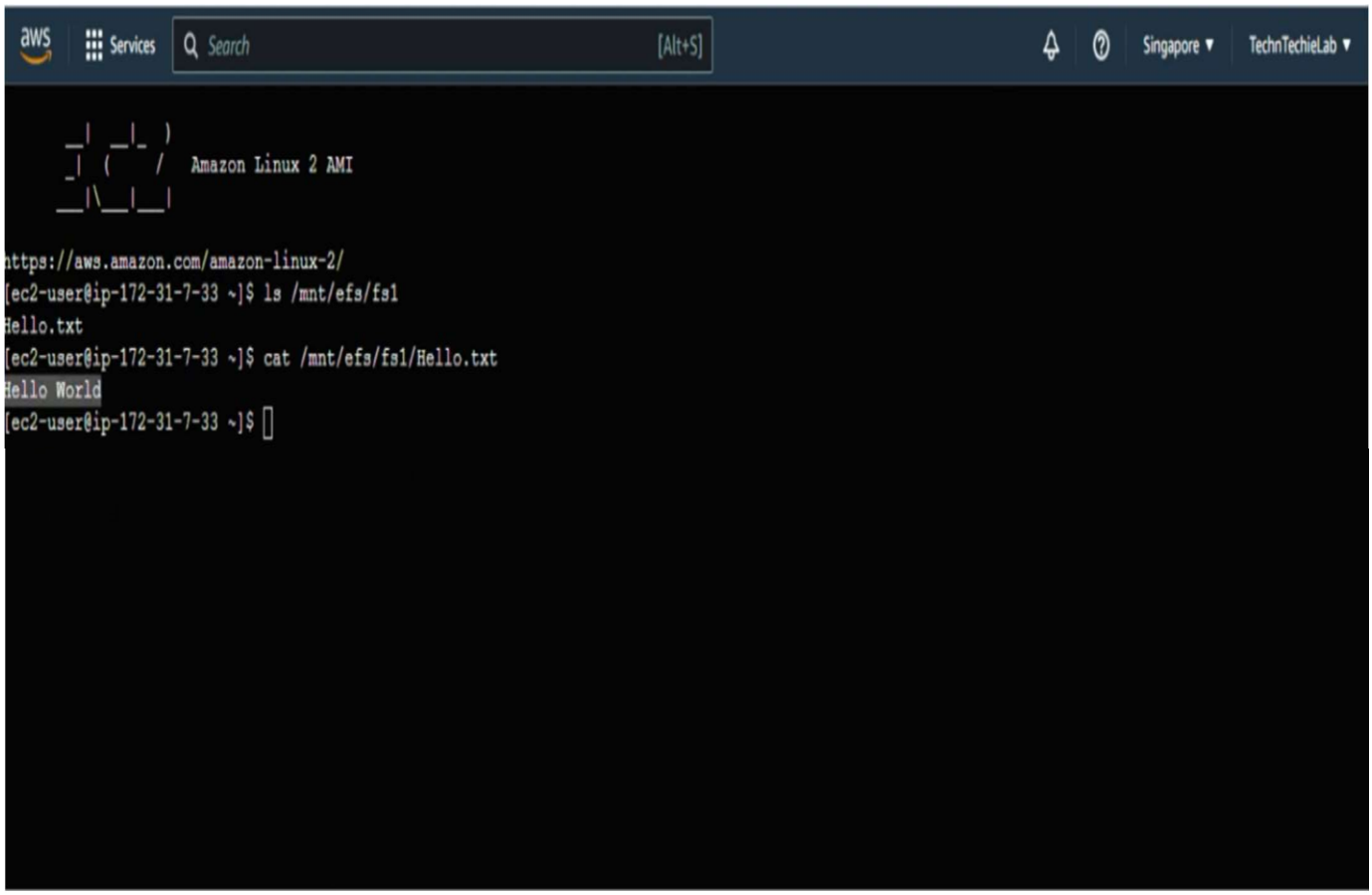
4. **Download a File from S3**

Retrieve a file using:

sh
Copy code
aws s3 cp s3://my-s3-bucket-dhanush/document.pdf /path/

OUTPUT:


New EC2 Experience
Tell us what you think
EC2 Dashboard
EC2 Global View
Events
Tags
Limits
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Instances (4) Info
Connect
Instance state
Actions
Launch instances
Find instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Alarm status
Availability Zon
First Instance
i-01bee14e7df746b6b
Stopped
t2.micro
-
No alarms
ap-southeast-1a
Instance3
i-0b5692bc9fc2af41a
Running
t2.micro
2/2 checks passed
No alarms
ap-southeast-1c
Instance2
i-0a7accf551a68cfa0
Running
t2.micro
2/2 checks passed
No alarms
ap-southeast-1a
Instance1
i-03ce281619011a9c1
Running
t2.micro
2/2 checks passed
No alarms
ap-southeast-1b
Select an instance


aws
Services
Search
[Alt+S]
Singapore
TechnTechieLab
Amazon Linux 2 AMI
https://aws.amazon.com/amazon-linux-2/
[ec2-user@ip-172-31-7-33 ~]$ ls /mnt/efs/fs1
Hello.txt
[ec2-user@ip-172-31-7-33 ~]$ cat /mnt/efs/fs1/Hello.txt
Hello World
[ec2-user@ip-172-31-7-33 ~]$

Ex.No:5	VPC CREATION
Date:	

AIM:

To create a **Virtual Private Cloud (VPC)** in AWS with a CIDR block of **120.0.0.0/16**, set up **one public subnet and two private subnets**, and configure a **NAT Gateway** to enable internet connectivity for private subnets.

PROCEDURE:

1. Understanding VPC Concepts

Virtual Private Cloud (VPC)

- A logically isolated network in AWS where you can launch resources like EC2 instances.

IP Address & CIDR (Classless Inter-Domain Routing)

- **Example CIDR:** 120.0.0.0/16
  - Provides **65,536 IP addresses** (120.0.0.0 – 120.0.255.255).
  - **Subnetting** divides this block into smaller networks.

Subnet & Subnet Mask

- **Public Subnet:** Directly accessible from the internet.
- **Private Subnet:** No direct internet access; requires a **NAT Gateway** for outbound access.

Internet Gateway vs NAT Gateway

- **Internet Gateway (IGW):** Allows public access to AWS resources.
- **NAT Gateway:** Allows private instances to access the internet while keeping them secure.

2. VPC and Subnet Setup

Step 1: Create a VPC

1. Go to **AWS Console → VPC Dashboard**.
2. Click **Create VPC**.
3. Set the following configurations:
   - **VPC Name:** MyVPC
   - **CIDR Block:** 120.0.0.0/16
   - **IPv6:** No
   - **Tenancy:** Default
4. Click **Create VPC**.

Step 2: Create Subnets

1. Go to the **Subnets** section and click **Create Subnet**.

   Create a Public Subnet
   - **Subnet Name:** Public-Subnet
   - **VPC:** Select MyVPC
   - **CIDR Block:** 120.0.1.0/24 *(Provides 256 IPs)*
   - **Enable Auto-Assign Public IPv4:** Yes
   - Click **Create**.

   Create Two Private Subnets
   - **Private Subnet 1:**
     - **Subnet Name:** Private-Subnet-1
     - **CIDR Block:** 120.0.2.0/24
   - **Private Subnet 2:**
     - **Subnet Name:** Private-Subnet-2
     - **CIDR Block:** 120.0.3.0/24
   - Click **Create** for both.

Step 3: Configure Internet Gateway

1. Go to the **Internet Gateways** section → Click **Create IGW**.
2. Enter **Name:** MyIGW → Click **Create**.
3. **Attach IGW to VPC:**
   - Select **MyVPC**.
   - Click **Attach**.

Step 4: Configure a Route Table for Public Subnet

1. Go to **Route Tables** → Click **Create Route Table**.
2. Enter **Name:** Public-RT
   - **VPC:** MyVPC
   - Click **Create**.
3. **Edit Routes:**
   - **Destination:** 0.0.0.0/0
   - **Target:** Internet Gateway (MyIGW)
   - Click **Save Changes**.
4. **Associate Public Subnet:**
   - Go to **Subnet Associations**.
   - Select **Public-Subnet**.
   - Click **Save**.

Step 5: Configure a NAT Gateway for Private Subnets

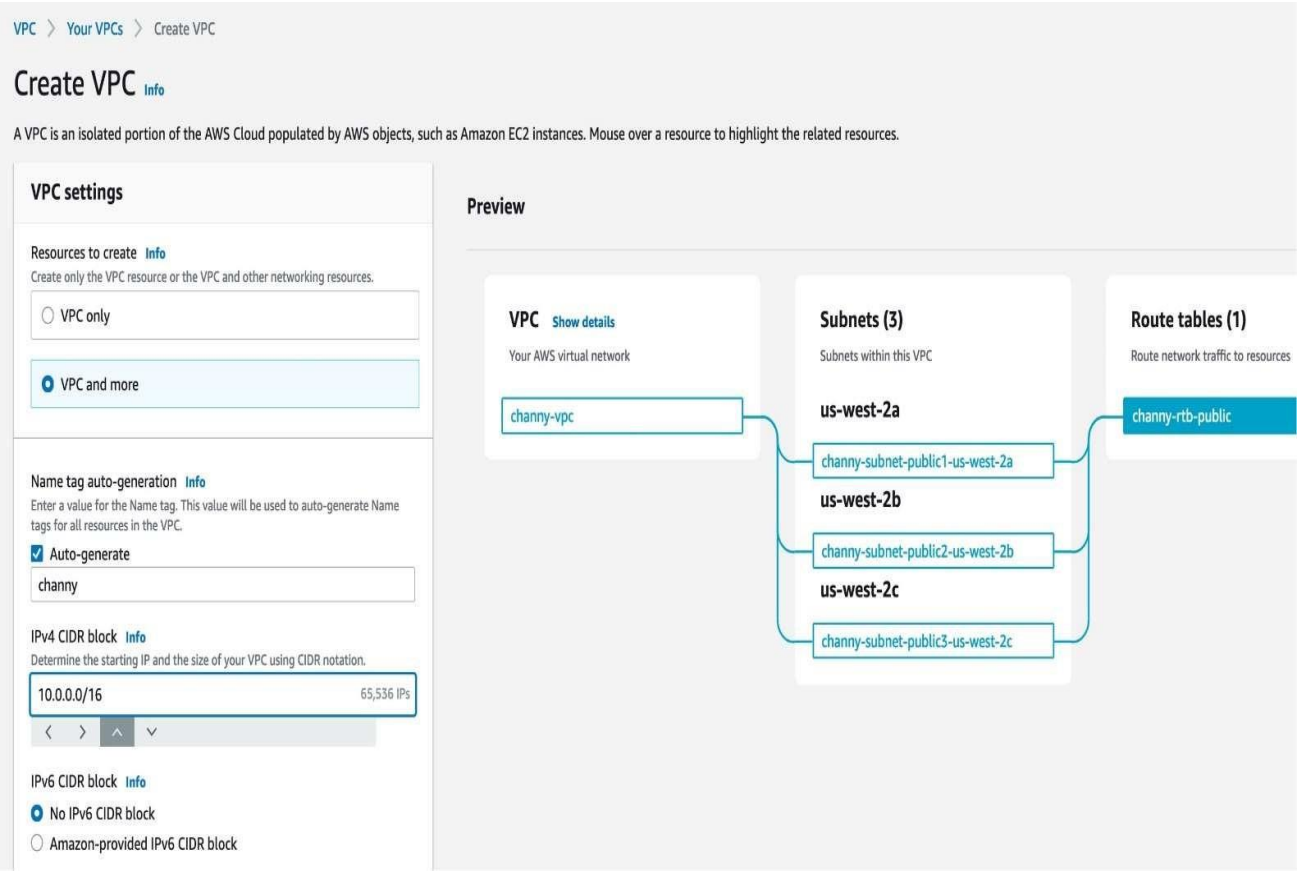
Go to **NAT Gateway** section → Click **Create NAT Gateway**.

1. **Set:**
   - **Name:** MyNATGateway
   - **Subnet:** Public-Subnet
   - **Elastic IP:** Allocate a new Elastic IP.
   - Click **Create NAT Gateway**.

2. **Create a Route Table for Private Subnets:**
    - **Name:** Private-RT
    - **VPC:** MyVPC
    - **Add Route:**
        - **Destination:** 0.0.0.0/0
        - **Target:** MyNATGateway
    - Click **Save**.

3. **Associate with Private Subnets:**
    - Go to **Subnet Associations**.
    - Select **Private-Subnet-1** & **Private-Subnet-2**.
    - Click **Save**.

OUTPUT:


VPC > Your VPCs > Create VPC
Create VPC Info
A VPC is an isolated portion of the AWS Cloud populated by AWS objects, such as Amazon EC2 instances. Mouse over a resource to highlight the related resources.
VPC settings
Resources to create Info
Create only the VPC resource or the VPC and other networking resources.
VPC only
VPC and more
Name tag auto-generation Info
Enter a value for the Name tag. This value will be used to auto-generate Name tags for all resources in the VPC.
Auto-generate
channy
IPv4 CIDR block Info
Determine the starting IP and the size of your VPC using CIDR notation.
10.0.0.0/16
65,536 IPs
IPv6 CIDR block Info
No IPv6 CIDR block
Amazon-provided IPv6 CIDR block
Preview
VPC Show details
Your AWS virtual network
channy-vpc
Subnets (3)
Subnets within this VPC
us-west-2a
channy-subnet-public1-us-west-2a
us-west-2b
channy-subnet-public2-us-west-2b
us-west-2c
channy-subnet-public3-us-west-2c
Route tables (1)
Route network traffic to resources
channy-rtb-public

Ex.No:6	IAM USER & ROLES
Date:	

AIM:

To create an **IAM policy** that restricts access to **EC2 only**, assign it to a new **IAM role**, create a new **IAM user with console access**, and assume the role **without signing out** to verify access.

PROCEDURE:

1. Understanding IAM Concepts

IAM (Identity and Access Management)

- A service in AWS that manages **users, groups, roles, and permissions**.

IAM Users vs Roles

- **User:** An entity with AWS credentials (**passwords, access keys**).
- **Role:** A **temporary permission set** that users or AWS services can assume.

IAM Policy

- **JSON-based rules** defining permissions.
- Example: **Restrict access to EC2 only**.

Assume Role

- Allows **switching to another role** without signing out.

2. Hands-On Implementation

Step 1: Create an IAM Policy to Restrict Access to EC2 Only

1. Go to **AWS Console → IAM → Policies**.
2. Click **Create Policy**.
3. Select **JSON Editor** and enter the following policy:

```
json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:*",
      "Resource": "*"
    },
    {
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
```

```
                "StringNotEquals": {
                    "aws:RequestedRegion": "us-east-1"
                }
            }
        }
    ]
}
```

4. Click **Next** → Name it **EC2-Only-Policy** → Click **Create Policy**.

Step 2: Create an IAM Role and Assign the Policy

1. Go to **AWS Console** → **IAM** → **Roles**.
2. Click **Create Role** → Select **AWS Service** → Choose **EC2**.
3. Click **Next: Permissions** → Search & Attach **EC2-Only-Policy**.
4. Click **Next** → Name it **EC2-Only-Role** → Click **Create Role**.

Step 3: Create an IAM User with Console Access

1. Go to **IAM** → **Users** → Click **Create User**.
2. Enter **User Name:** EC2-User.
3. Check **AWS Management Console Access**.
4. **Set Password:** Auto-generated / Custom.
5. Click **Next: Permissions** → Select **Attach existing policies directly**.
6. Attach **EC2-Only-Policy**.
7. Click **Create User**.

Step 4: Assume Role Without Signing Out

Using AWS Console:

1. Go to **IAM** → **Roles** → Select **EC2-Only-Role**.
2. Copy the **Role ARN**.
3. Open **AWS Console**.
4. Go to **IAM** → **Your Username** → **Switch Role**.
5. Enter **EC2-Only-Role** and click **Switch Role**.
6. **Verify EC2 access** by trying to access other services (**like S3**), which should be restricted.

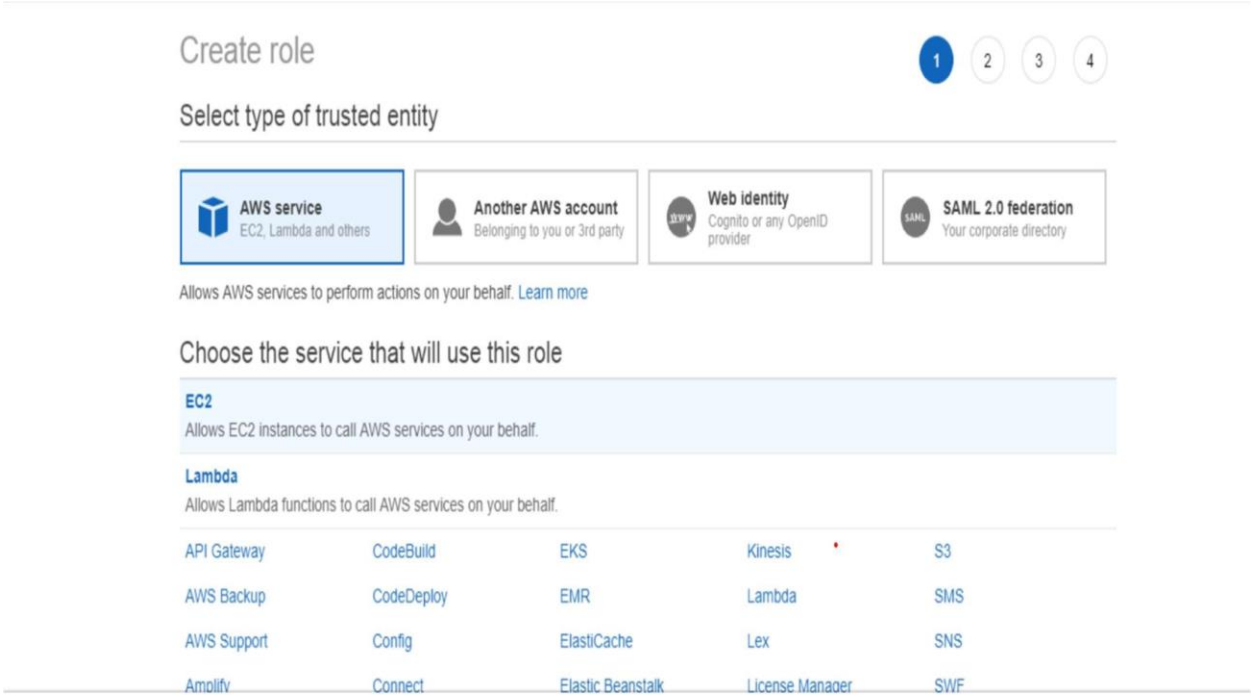
Using AWS CLI:

1. Run the following command to assume the role:

   sh
   Copy code
   aws sts assume-role --role-arn "arn:aws:iam::<ACCOUNT_ID>:role/EC2-Only-Role" --role-session-name EC2Session

2. Verify that **EC2 access works**, but other services (like S3) are **restricted**.

OUTPUT:


Create role
1
2
3
4
Select type of trusted entity
AWS service
EC2, Lambda and others
Another AWS account
Belonging to you or 3rd party
Web identity
Cognito or any OpenID provider
SAML 2.0 federation
Your corporate directory
Allows AWS services to perform actions on your behalf. Learn more
Choose the service that will use this role
EC2
Allows EC2 instances to call AWS services on your behalf.
Lambda
Allows Lambda functions to call AWS services on your behalf.
API Gateway
CodeBuild
EKS
Kinesis
S3
AWS Backup
CodeDeploy
EMR
Lambda
SMS
AWS Support
Config
ElastiCache
Lex
SNS
Amplify
Connect
Elastic Beanstalk
License Manager
SWF

Ex.No:7	RDS CREATION
Date:	

AIM:

To create an **RDS MySQL database instance**, connect it to an **EC2 instance**, and perform **basic SQL tasks**.

PROCEDURE:

1. Understanding RDS Concepts

AWS RDS (Relational Database Service)

- A **managed service** for relational databases like **MySQL, PostgreSQL, SQL Server**, etc.
- Handles **automatic backups, scaling, and maintenance**.

Database Instance

- A **running database** in AWS RDS.

Storage

- **Allocated space** for the database (**SSD-backed storage**).

Connecting to EC2

- The database can be accessed from an **EC2 instance via MySQL client**.

2. Hands-On Implementation

Step 1: Create an EC2 Instance (If Not Already Available)

1. Go to **AWS Console → EC2** → Click **Launch Instance**.
2. Select **Ubuntu** or **Amazon Linux**.
3. Choose **t2.micro** (**Free Tier**).
4. **Configure network settings**:
   - Ensure it's in the **same VPC** as RDS.
   - Assign a **Public IP**.
5. **Add Security Group**:
   - **Allow SSH** (**Port 22**) for remote access.
   - **Allow MySQL/Aurora** (**Port 3306**) for database connectivity.
6. Click **Launch Instance** and **connect via SSH**.

Step 2: Create an RDS MySQL Database Instance

1. Go to **AWS Console → RDS** → Click **Create Database**.
2. Select **Standard Create**.
3. Choose **MySQL** as the engine.
4. Select **Free Tier** (**db.t3.micro**).
5. **Set database credentials**:
   - **Username:** admin
   - **Password:** yourpassword

6. **Configure network settings**:
    - Ensure it's in the **same VPC** as the EC2 instance.
    - **Enable Public Access** (optional, if accessing from outside AWS).
    - Assign a **Security Group** that allows inbound **MySQL (Port 3306)**.
7. Click **Create Database** and **wait** for the instance to launch.

Step 3: Modify RDS Security Group to Allow EC2 Access

1. Go to **AWS Console → EC2 → Security Groups**.
2. Select the **RDS Security Group**.
3. **Add an Inbound Rule**:
    - **Type:** MySQL/Aurora
    - **Port:** 3306
    - **Source: EC2 Security Group**
4. Click **Save Changes**.

Step 4: Connect to MySQL from EC2 Instance

1. **SSH into EC2 Instance**:
    sh
    Copy code
    ssh -i your-key.pem ubuntu@your-ec2-public-ip

2. **Install MySQL client (if not installed)**:
    sh
    Copy code
    sudo apt update
    sudo apt install mysql-client -y

3. **Connect to RDS MySQL Database**:
    sh
    Copy code
    mysql -h your-rds-endpoint -u admin -p

4. **Enter the password** when prompted.

Step 5: Perform Basic SQL Tasks

1. **Create a Database**:
    sql
    Copy code
    CREATE DATABASE testdb;

2. **Use the Database**:
    sql
    Copy code
    USE testdb;

3. **Create a Table**:
    sql
    Copy code

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE
);
```

4. **Insert Data**:

   sql
   Copy code
   INSERT INTO users (name, email) VALUES ('John Doe', 'john@example.com');

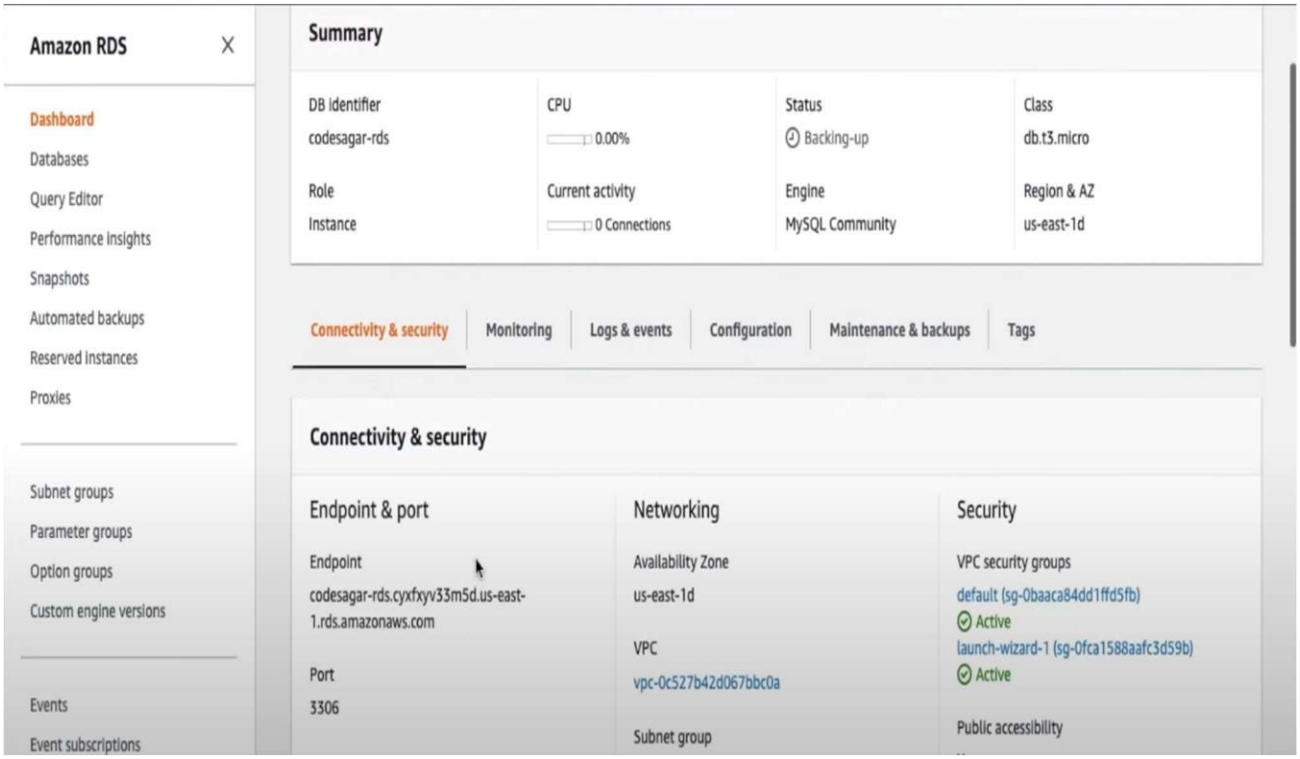
5. **View Data**:

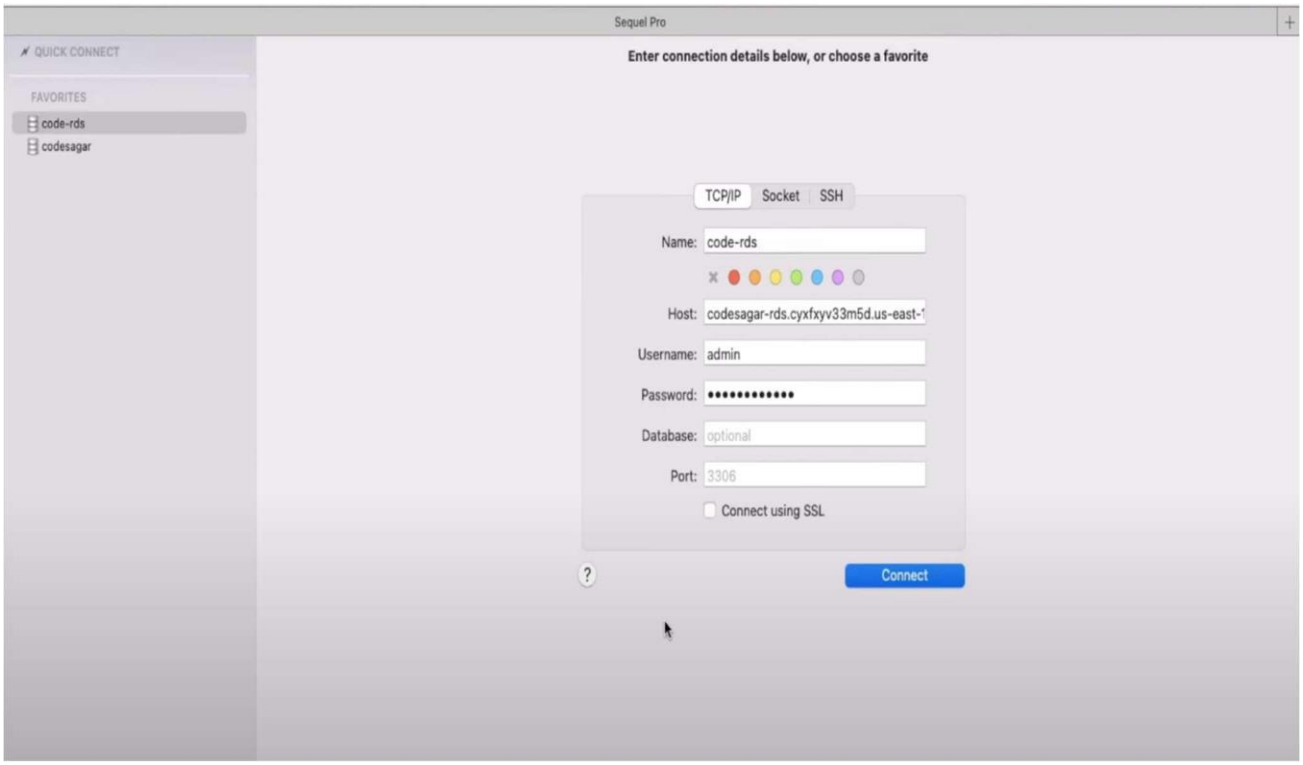
   sql
   Copy code
   SELECT * FROM users;

6. **Exit MySQL**:

   sql
   Copy code
   EXIT;

OUTPUT:


Amazon RDS
Dashboard
Databases
Query Editor
Performance insights
Snapshots
Automated backups
Reserved instances
Proxies
Subnet groups
Parameter groups
Option groups
Custom engine versions
Events
Event subscriptions
Summary
DB identifier
codesagar-rds
CPU
0.00%
Status
Backing-up
Class
db.t3.micro
Role
Instance
Current activity
0 Connections
Engine
MySQL Community
Region & AZ
us-east-1d
Connectivity & security
Monitoring
Logs & events
Configuration
Maintenance & backups
Tags
Connectivity & security
Endpoint & port
Endpoint
codesagar-rds.cyxfxyv33m5d.us-east-1.rds.amazonaws.com
Port
3306
Networking
Availability Zone
us-east-1d
VPC
vpc-0c527b42d067bbc0a
Subnet group
Security
VPC security groups
default (sg-0baaca84dd1ffd5fb)
Active
launch-wizard-1 (sg-0fca1588aafc3d59b)
Active
Public accessibility


Sequel Pro
QUICK CONNECT
FAVORITES
code-rds
codesagar
Enter connection details below, or choose a favorite
TCP/IP
Socket
SSH
Name: code-rds
Host: codesagar-rds.cyxfxyv33m5d.us-east-1
Username: admin
Password: ••••••••••••
Database: optional
Port: 3306
Connect using SSL
Connect